

Bom, eu tenho um grupo de trabalho da matéria de arquitetura e desenho de software e eu fiquei com a parte da engenharia reversa.

Nosso projeto vai ser um jogo tamagochi, essa é a ideia. Consegue me ajudar a fazer esses artefatos de engenharia reversa?

Thought for 30s

Claro, posso te ajudar bastante com isso! Pelo que vi no site da disciplina, o módulo de Engenharia Reversa trabalha em cima da ideia de partir de um produto de software **já existente** (a interface gráfica dele, os fluxos de tela, etc.) para recuperar requisitos e depois modelar isso em BPMN.

Então, antes de montar os artefatos, preciso entender um detalhe: para fazer engenharia reversa de "algo", geralmente se escolhe um app/jogo real já existente como referência (não o produto que vocês ainda vão construir, já que ele ainda não existe).

Vocês já escolheram um app/jogo de tamagotchi existente para usar como referência na engenharia reversa?

Ainda não escolhemos

Searched the web

Boa, então vamos escolher uma referência. Pesquisei algumas opções de jogos de tamagotchi que rodam direto no navegador — isso é importante porque o material da disciplina sugere usar as Ferramentas de Desenvolvedor do Chrome/Firefox (aba Network/Sources) pra inspecionar telas, CSS e fluxos, o que só funciona bem em apps web (não em apps nativos de celular).

Aqui vão 3 opções com escopos diferentes:

1. **Tamagotchi (leemallon.com)** — escopo enxuto: alimentar, brincar, dar banho, ciclo dia/noite, salvamento persistente. Bom pra documentar com profundidade sem virar um trabalho gigante.
2. **Tamaweb** — bem mais completo (minigames, relacionamentos, casa, "underworld"...). Rico, mas pode ser trabalho demais pra um artefato de disciplina.
3. **Tamagotchi Online (browsergameshq.com)** — bem clássico e simples: alimentar, limpar, cuidar.

Coloquei, teste e faça o que deve ser feito

List_connected_browsers

Encontrei um navegador conectado. Qual devo usar?

Browser 1 (593cbe77-5cf2-40ec-84a7-1965f53dfb2e)

Used 49 tools, loaded tools · 5 notes



Cheguei no limite de ferramentas por enquanto — aqui vai um resumo do que já mapeei no app (<https://tamagochi.leemallon.com/>) até agora:

Telas identificadas:

1. **Nomear pet** — modal "NAME YOUR PET" com campo de texto (default "Tama") + botão OK
2. **Tela principal (idle)** — dispositivo estilo Tamagotchi/GameBoy: LCD com o sprite do pet, D-pad, botão A (vermelho) e B (cinza)
3. **Menu principal** (abre com A) — 3 ícones: **FEED**, **ACTION**, **STATUS**
4. **Submenu FEED** — MEAL, SNACK, MEDS
5. **Tela STATUS** — nome do pet + barras: HUN, HAP, HLT, NRG, CLN, AGE (em minutos)
6. **Submenu ACTION** — PLAY, BATH, STARS, LGT OFF (ainda não testei este último)
7. **Minijogo STARS** — "STARS: 0 / TIME: 9", instrução "TAP THE STARS", estrelas aparecem em posições aleatórias pra clicar (ainda terminando de testar o resultado final)

Regras de negócio já confirmadas (testadas ao vivo, com valores antes/depois):

- **MEAL** → aumenta HUN (+21)
- **SNACK** → aumenta HAP (+15)
- **MEDS** → aumenta HLT (+5)
- **PLAY** → aumenta HAP (+11), **mas reduz NRG (-10)** — trade-off interessante
- **BATH** → aumenta CLN (+36)
- Todos os stats **decaem naturalmente com o tempo** (a cada "tick")
- Navegação: **A** = abrir menu/confirmar · **B** = voltar um nível · D-pad ↑↓ = navega lista (circular) · D-pad ←→ = alterna entre ícones do menu principal

Ainda falta: testar LGT OFF, terminar o minigame STARS pra ver o resultado final, e confirmar se existe estado de "pet doente/morto" por negligência.

Continue

Used 8 tools · 2 notes



Vou dar uma espiada no código-fonte (JS) da página pra confirmar regras que não dá pra ver só brincando — como o que acontece quando os stats chegam a zero, se existe evolução do pet, etc. Isso é exatamente a técnica de inspeção via "Sources"/"Network" que o material da disciplina sugere.

Used 40 tools, viewed a file, ran a command · 3 notes

Beleza, cheguei no limite de ferramentas por agora — mas o levantamento já está muito sólido. Aqui vai o resumo completo do que descobri até agora sobre o tamagochi.leemallon.com, incluindo dados confirmados diretamente no código-fonte (não só por clicar na interface):

Telas e elementos mapeados

1. Nomear pet (modal) — campo texto (default "Tama") + botão OK
2. Tela principal (idle) — sprite do pet, D-pad, botão A (vermelho) e B (cinza)
3. Menu principal — FEED / ACTION / STATUS
4. Submenu FEED — MEAL / SNACK / MEDS
5. Submenu ACTION — PLAY / BATH / STARS / LGT OFF↔LGT ON (toggle)
6. Tela STATUS — barras HUN/HAP/HLT/NRG/CLN + AGE
7. Overlay de Atenção — aviso com som quando o pet precisa de algo
8. Overlay de Desmaio (FAINTED) — quando HLT chega a 0; pressionar A revive o pet
9. Minigame STARS — 10s de duração, estrelas somem em -2,5s, resultado final "CAUGHT N STARS!"
10. Notificação "You were away for N minutes" ao voltar pra aba

Regras de negócio confirmadas no código (pet.js, ui.js, renderer.js)

- MEAL: HUN +30, HLT +5, CLN -3
- SNACK: HUN +10, HAP +15, HLT -5, CLN -5
- MEDS: HLT +25, HAP -10
- PLAY: HAP +20, NRG -10, CLN -5
- BATH: CLN +40, HAP +5, NRG -5
- STARS: HAP += min(estrelas×5, 40), NRG -8
- Decaimento natural por segundo: fome (1/30), felicidade (1/45), saúde (1/60, multiplicada se fome/felicidade/limpeza baixas), energia (1/40, varia com período do dia e luz apagada), limpeza (1/50)
- Sono automático: energia chega a 0 → pet dorme sozinho e recupera energia; acorda ao passar de 80
- Desmaio: saúde chega a 0 → isFainted=true → só revive com botão A (reseta tudo pra 50)
- Sistema de atenção (prioridade): fome<15 → "hungry"; felicidade<15 → "lonely"; saúde<25 → "sick"; limpeza<15 → "dirty"
- Ações são bloqueadas se o pet estiver desmaiado ou dormindo

Isso me dá base sólida — verificada, não só suposta — pra montar os 4 artefatos que a disciplina pede (lista de telas, tabela de transições, regras de negócio documentadas e o diagrama BPMN).